

Pointer Analysis (1)

CSE552 Program Analysis — Lecture 14

Jaemin Hong

Motivation

```
*x = 42;  
*y = -87;  
z = *x;
```

- ▶ The value of z depends on whether x and y are aliases or not

Allocation-Site Abstraction

- ▶ A program can have infinitely many memory cells (memory locations) at runtime
- ▶ We introduce
 - ▶ an abstract cell x for every program variable x
 - ▶ an abstract cell $\text{alloc-}i$ for each occurrence of an `alloc` operation in the program, where i is a unique index
- ▶ Each abstract cell represents the set of cells at runtime that are allocated at the corresponding source location
- ▶ We use Cell to denote the set of abstract cells for the given program

Points-To Analysis

- ▶ Points-to analysis determines the set of abstract memory cells that each pointer variable may point to
 - ▶ computes a function $pt : Var \rightarrow \mathcal{P}(Cell)$
- ▶ Points-to information can be used to approximate facts related to pointers
 - ▶ e.g., x and y may be aliases if $pt(x) \cap pt(y) \neq \emptyset$

Andersen-Style Analysis¹ — Nodes and Constraint Variables

$$\begin{array}{lcl} \text{Node } v & ::= & x = \text{alloc}() \\ & | & x = \&y \\ & | & x = y \\ & | & x = *y \\ & | & *x = y \end{array}$$

- For each abstract cell c , we introduce a constraint variable $\llbracket c \rrbracket$, which ranges over sets of abstract cells

¹Program analysis and specialization for the C programming language
(Andersen, 1994)

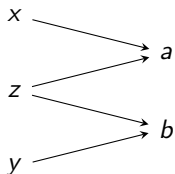
Andersen-Style Analysis — Constraint Rules

- ▶ $x = \text{alloc}()$: $\text{alloc-}i \in \llbracket x \rrbracket$
- ▶ $x = \&y$: $y \in \llbracket x \rrbracket$
- ▶ $x = y$: $\llbracket y \rrbracket \subseteq \llbracket x \rrbracket$
- ▶ $x = *y$: $\forall c \in \text{Cell}. c \in \llbracket y \rrbracket \Rightarrow \llbracket c \rrbracket \subseteq \llbracket x \rrbracket$
- ▶ $*x = y$: $\forall c \in \text{Cell}. c \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket c \rrbracket$

- ▶ Constraints can be solved using the cubic algorithm
- ▶ $pt(x) = \llbracket x \rrbracket$

Andersen-Style Analysis — Example 1

```
x = &a;  
y = &b;  
if ... {  
    z = x;  
} else {  
    z = y;  
}
```



Constraints:

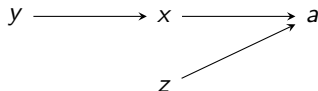
- ▶ $x = \&a$: $a \in \llbracket x \rrbracket$
- ▶ $y = \&b$: $b \in \llbracket y \rrbracket$
- ▶ $z = x$: $\llbracket x \rrbracket \subseteq \llbracket z \rrbracket$
- ▶ $z = y$: $\llbracket y \rrbracket \subseteq \llbracket z \rrbracket$

Solutions:

- ▶ $\llbracket x \rrbracket = \{a\}$
- ▶ $\llbracket y \rrbracket = \{b\}$
- ▶ $\llbracket z \rrbracket = \{a, b\}$

Andersen-Style Analysis — Example 2

```
x = &a;  
y = &x;  
z = *y;
```



Constraints:

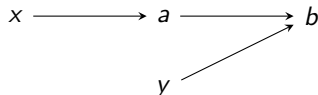
- ▶ $x = \&a: a \in \llbracket x \rrbracket$
- ▶ $y = \&x: x \in \llbracket y \rrbracket$
- ▶ $z = *y: x \in \llbracket y \rrbracket \Rightarrow \llbracket x \rrbracket \subseteq \llbracket z \rrbracket \wedge$
 $y \in \llbracket y \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket z \rrbracket \wedge$
 $z \in \llbracket y \rrbracket \Rightarrow \llbracket z \rrbracket \subseteq \llbracket z \rrbracket \wedge$
 $a \in \llbracket y \rrbracket \Rightarrow \llbracket a \rrbracket \subseteq \llbracket z \rrbracket$

Solutions:

- ▶ $\llbracket x \rrbracket = \{a\}$
- ▶ $\llbracket y \rrbracket = \{x\}$
- ▶ $\llbracket z \rrbracket = \{a\}$

Andersen-Style Analysis — Example 3

```
x = &a;  
y = &b;  
*x = y;
```



Constraints:

- ▶ $x = \&a: a \in \llbracket x \rrbracket$
- ▶ $y = \&b: b \in \llbracket y \rrbracket$
- ▶ $*x = y: x \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket x \rrbracket \wedge$
 $y \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket y \rrbracket \wedge$
 $a \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket a \rrbracket \wedge$
 $b \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket b \rrbracket$

Solutions:

- ▶ $\llbracket x \rrbracket = \{a\}$
- ▶ $\llbracket y \rrbracket = \{b\}$
- ▶ $\llbracket a \rrbracket = \{b\}$

Andersen-Style Analysis — Example 4

```
x = alloc(); // 0
y = alloc(); // 1
*x = y;
```

$x \longrightarrow \text{alloc-0} \longrightarrow \text{alloc-1}$



Constraints:

- ▶ $x = \text{alloc}()$: $\text{alloc-0} \in \llbracket x \rrbracket$
- ▶ $y = \text{alloc}()$: $\text{alloc-1} \in \llbracket y \rrbracket$
- ▶ $*x = y$: $x \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket x \rrbracket \wedge$
 $y \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket y \rrbracket \wedge$
 $\text{alloc-0} \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket \text{alloc-0} \rrbracket \wedge$
 $\text{alloc-1} \in \llbracket x \rrbracket \Rightarrow \llbracket y \rrbracket \subseteq \llbracket \text{alloc-1} \rrbracket$

Solutions:

- ▶ $\llbracket x \rrbracket = \{\text{alloc-0}\}$
- ▶ $\llbracket y \rrbracket = \{\text{alloc-1}\}$
- ▶ $\llbracket \text{alloc-0} \rrbracket = \{\text{alloc-1}\}$
- ▶ $\llbracket \text{alloc-1} \rrbracket = \emptyset$

Steensgaard-Style Analysis² — Characteristics

- ▶ Views assignments as being bidirectional
- ▶ Less precise but more efficient than Andersen-style analysis

²Points-to analysis in almost linear time (Steensgaard, 1996)

Steensgaard-Style Analysis — Constraint Rules

- ▶ For each abstract cell c , we introduce a term variable $\llbracket c \rrbracket$
- ▶ $\&$ is the only term constructor
- ▶ $x = \text{alloc}()$: $\llbracket x \rrbracket = \&\llbracket \text{alloc-}i \rrbracket$
- ▶ $x = \&y$: $\llbracket x \rrbracket = \&\llbracket y \rrbracket$
- ▶ $x = y$: $\llbracket x \rrbracket = \llbracket y \rrbracket$
- ▶ $x = *y$: $\llbracket y \rrbracket = \&\llbracket x \rrbracket$
- ▶ $*x = y$: $\llbracket x \rrbracket = \&\llbracket y \rrbracket$

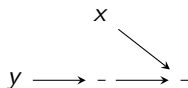
Steensgaard-Style Analysis — Intuition

- ▶ $\llbracket x \rrbracket = \&\llbracket y \rrbracket$ intuitively means x points to y

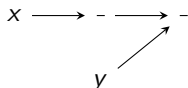
$$x = \&y: \llbracket x \rrbracket = \&\llbracket y \rrbracket$$

$$x \longrightarrow y$$

$$x = *y: \llbracket y \rrbracket = \&\llbracket x \rrbracket$$



$$*x = y: \llbracket x \rrbracket = \&\llbracket y \rrbracket$$



Steensgaard-Style Analysis — Constraint Solving

- ▶ Constraints can be solved using the unification algorithm
- ▶ $pt(x) = \{c \mid \llbracket x \rrbracket = \&\llbracket c \rrbracket\}$

Steensgaard-Style Analysis — Example 1

```
x = &a;  
y = &b;  
if ... {  
    z = x;  
} else {  
    z = y;  
}
```

$x, y, z \longrightarrow a, b$

Constraints:

- ▶ $x = \&a: \llbracket x \rrbracket = \&\llbracket a \rrbracket$
- ▶ $y = \&b: \llbracket y \rrbracket = \&\llbracket b \rrbracket$
- ▶ $z = x: \llbracket z \rrbracket = \llbracket x \rrbracket$
- ▶ $z = y: \llbracket z \rrbracket = \llbracket y \rrbracket$

Solutions:

- ▶ $\llbracket z \rrbracket = \llbracket x \rrbracket = \llbracket y \rrbracket = \&\llbracket a \rrbracket = \&\llbracket b \rrbracket$
- ▶ $\llbracket a \rrbracket = \llbracket b \rrbracket$
- ▶ $pt(z) = pt(x) = pt(y) = \{a, b\}$

Steensgaard-Style Analysis — Example 1 vs. Andersen

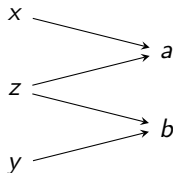
Steensgaard-style analysis

- ▶ $pt(z) = pt(x) = pt(y) = \{a, b\}$

$x, y, z \longrightarrow a, b$

Andersen-style analysis

- ▶ $pt(z) = \{a, b\}$
- ▶ $pt(x) = \{a\}$
- ▶ $pt(y) = \{b\}$



Steensgaard-Style Analysis — Example 2

```
x = &a;  
y = &x;  
z = *y;
```

$y \longrightarrow x, z \longrightarrow a$

Constraints:

- ▶ $x = \&a: \llbracket x \rrbracket = \&\llbracket a \rrbracket$
- ▶ $y = \&x: \llbracket y \rrbracket = \&\llbracket x \rrbracket$
- ▶ $z = *y: \llbracket y \rrbracket = \&\llbracket z \rrbracket$

Solutions:

- ▶ $\llbracket y \rrbracket = \&\llbracket x \rrbracket = \&\llbracket z \rrbracket$
- ▶ $\llbracket x \rrbracket = \llbracket z \rrbracket = \&\llbracket a \rrbracket$
- ▶ $pt(y) = \{x, z\}$
- ▶ $pt(x) = pt(z) = \{a\}$

Steensgaard-Style Analysis — Example 2 vs. Andersen

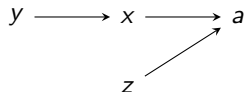
Steensgaard-style analysis

- ▶ $pt(y) = \{x, z\}$
- ▶ $pt(x) = pt(z) = \{a\}$

$y \longrightarrow x, z \longrightarrow a$

Andersen-style analysis

- ▶ $pt(x) = \{a\}$
- ▶ $pt(y) = \{x\}$
- ▶ $pt(z) = \{a\}$



Steensgaard-Style Analysis — Example 3

```
x = &a;  
y = &b;  
*x = y;
```

$x \longrightarrow a, y \longrightarrow b$

Constraints:

- ▶ $x = \&a: \llbracket x \rrbracket = \&\llbracket a \rrbracket$
- ▶ $y = \&b: \llbracket y \rrbracket = \&\llbracket b \rrbracket$
- ▶ $*x = y: \llbracket x \rrbracket = \&\llbracket y \rrbracket$

Solutions:

- ▶ $\llbracket x \rrbracket = \&\llbracket y \rrbracket = \&\llbracket a \rrbracket$
- ▶ $\llbracket y \rrbracket = \llbracket a \rrbracket = \&\llbracket b \rrbracket$
- ▶ $pt(x) = \{a, y\}$
- ▶ $pt(y) = pt(a) = \{b\}$

Steensgaard-Style Analysis — Example 3 vs. Andersen

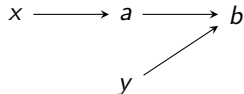
Steensgaard-style analysis

- ▶ $pt(x) = \{a, y\}$
- ▶ $pt(z) = pt(y) = \{b\}$

$x \longrightarrow a, y \longrightarrow b$

Andersen-style analysis

- ▶ $pt(x) = \{a\}$
- ▶ $pt(y) = \{b\}$
- ▶ $pt(a) = \{b\}$



Interprocedural Pointer Analysis

```
*x = f;  
(*y)(z);
```

```
fn f(a) { *a = &y; }  
fn g(b) { *b = &z; }  
h(&x);
```

- ▶ Deciding control flow requires points-to information
- ▶ Deciding points-to information requires control flow information
- ▶ We need to perform points-to analysis and control flow analysis simultaneously when we have pointers and function pointers

Andersen-Style Analysis

- ▶ Can be directly combined with the control flow analysis
- ▶ $\text{fn } f(\dots) \{ \dots \}: f \in \llbracket f \rrbracket$
- ▶ $x = y(z_1, \dots, z_n):$
 $\forall f. f \in \llbracket y \rrbracket \Rightarrow (\llbracket z_1 \rrbracket \subseteq \llbracket a_f^1 \rrbracket \wedge \dots \wedge \llbracket z_n \rrbracket \subseteq \llbracket a_f^n \rrbracket \wedge \llbracket RET_f \rrbracket \subseteq \llbracket x \rrbracket)$

Andersen-Style Analysis — Example

```
fn f(a) {}  
fn g(b) {}  
  
x = f;  
y = g;  
if ... {  
  z = &x;  
} else {  
  z = &y;  
}  
w = *z;  
u = &v;  
w(u);
```

Constraints:

- ▶ $\text{fn } f: f \in \llbracket f \rrbracket$
- ▶ $\text{fn } g: g \in \llbracket g \rrbracket$
- ▶ $x = f: \llbracket f \rrbracket \subseteq \llbracket x \rrbracket$
- ▶ $y = g: \llbracket g \rrbracket \subseteq \llbracket y \rrbracket$
- ▶ $z = \&x: x \in \llbracket z \rrbracket$
- ▶ $z = \&y: y \in \llbracket z \rrbracket$
- ▶ $w = *z: \forall c. c \in \llbracket z \rrbracket \Rightarrow \llbracket c \rrbracket \subseteq \llbracket w \rrbracket$
- ▶ $u = \&v: v \in \llbracket u \rrbracket$
- ▶ $w(u): \forall f. f \in \llbracket w \rrbracket \Rightarrow \llbracket u \rrbracket \subseteq \llbracket a_f^1 \rrbracket$

Solutions:

- ▶ $\llbracket f \rrbracket = \{f\}, \llbracket g \rrbracket = \{g\}$
- ▶ $\llbracket x \rrbracket = \{f\}, \llbracket y \rrbracket = \{g\}$
- ▶ $\llbracket z \rrbracket = \{x, y\}, \llbracket w \rrbracket = \{f, g\}$
- ▶ $\llbracket u \rrbracket = \{v\}, \llbracket a \rrbracket = \{v\}, \llbracket b \rrbracket = \{v\}$

Steensgaard-Style Analysis

- ▶ The term constructors represent both variable pointers and function pointers

$$\llbracket x \rrbracket = (\&\llbracket y \rrbracket, \text{fn}(\llbracket z_1 \rrbracket, \dots, \llbracket z_n \rrbracket) \rightarrow \llbracket w \rrbracket)$$

- ▶ $x = \text{alloc}()$: $\llbracket x \rrbracket = (\&\llbracket \text{alloc-}i \rrbracket, _)$
- ▶ $x = \&y$: $\llbracket x \rrbracket = (\&\llbracket y \rrbracket, _)$
- ▶ $x = y$: $\llbracket x \rrbracket = \llbracket y \rrbracket$
- ▶ $x = *y$: $\llbracket y \rrbracket = (\&\llbracket x \rrbracket, _)$
- ▶ $*x = y$: $\llbracket x \rrbracket = (\&\llbracket y \rrbracket, _)$
- ▶ $\text{fn } f(x_1, \dots, x_n) \{ \text{return } y; \}$: $\llbracket f \rrbracket = (_, \text{fn}(\llbracket x_1 \rrbracket, \dots, \llbracket x_n \rrbracket) \rightarrow \llbracket y \rrbracket)$
- ▶ $x = y(z_1, \dots, z_n)$: $\llbracket y \rrbracket = (_, \text{fn}(\llbracket z_1 \rrbracket, \dots, \llbracket z_n \rrbracket) \rightarrow \llbracket x \rrbracket)$

Steensgaard-Style Analysis — Example

```
fn f(a) {}  
fn g(b) {}  
  
x = f;  
y = g;  
if ... {  
  z = &x;  
} else {  
  z = &y;  
}  
w = *z;  
u = &v;  
w(u);
```

Constraints:

- ▶ $\llbracket f \rrbracket = (_, \text{fn}(\llbracket a \rrbracket))$
- ▶ $\llbracket g \rrbracket = (_, \text{fn}(\llbracket b \rrbracket))$
- ▶ $x = f: \llbracket x \rrbracket = \llbracket f \rrbracket$
- ▶ $y = g: \llbracket y \rrbracket = \llbracket g \rrbracket$
- ▶ $z = \&x: \llbracket z \rrbracket = (\&\llbracket x \rrbracket, _)$
- ▶ $z = \&y: \llbracket z \rrbracket = (\&\llbracket y \rrbracket, _)$
- ▶ $w = *z: \llbracket z \rrbracket = (\&\llbracket w \rrbracket, _)$
- ▶ $u = \&v: \llbracket u \rrbracket = (\&\llbracket v \rrbracket, _)$
- ▶ $w(u): \llbracket w \rrbracket = (_, \text{fn}(\llbracket u \rrbracket))$

Solutions:

- ▶ $\llbracket f \rrbracket = \llbracket g \rrbracket = \llbracket x \rrbracket = \llbracket y \rrbracket = \llbracket w \rrbracket = (_, \text{fn}(\llbracket a \rrbracket))$
- ▶ $\llbracket z \rrbracket = (\&\llbracket x \rrbracket, _)$
- ▶ $\llbracket u \rrbracket = \llbracket a \rrbracket = \llbracket b \rrbracket = (\&\llbracket v \rrbracket, _)$
- ▶ $\text{func}(f) = \text{func}(g) = \text{func}(x) = \text{func}(y) = \text{func}(w) = \{f, g\}$
- ▶ $\text{pt}(z) = \{x, y\}$
- ▶ $\text{pt}(u) = \text{pt}(a) = \text{pt}(b) = \{v\}$

Summary

- ▶ Allocation-site abstraction lets us reason about unbounded memory with a finite set of abstract cells
- ▶ Andersen-style analysis encodes pointer statements as subset constraints over $\llbracket c \rrbracket$ and solves them with the cubic algorithm
- ▶ Steensgaard-style analysis treats assignments bidirectionally, uses term equalities with the $\&$ constructor, and solves them via unification — faster but less precise
- ▶ With function pointers, control flow and points-to information are mutually dependent and must be computed together